

ECE 278C - Imaging Systems

Assignment 8: Short Baseline Geolocation Systems (Approximation techniques)

Name: Xinyi Yang

Date: December 5, 2025

1. Introduction

This assignment implements two approximation techniques for target geolocation using the short baseline bistatic radar system from Assignment 7. The system consists of one central transmitter and two receivers separated by baseline distance 2D. The goal is to reconstruct the target's position using approximate methods and analyze their effectiveness compared to the exact bistatic backward propagation.

Key features of the simulation:

- 1) Generation of received signals $g_1(n)$ and $g_2(n)$ from a single point target.
- 2) Approximation-based imaging: monostatic model and range-bearing polar model.
- 3) Video generation showing frequency convergence over N steps.
- 4) Parameter sweeps to evaluate localization error with respect to:
 - Transmitter-receiver separation 2D
 - Bandwidth (via center frequency f_0)
 - Number of frequency steps N
 - Target range r_0
 - Target bearing angle θ_0

2. System Configuration

2.1 Geometry and Signal Model

The system geometry is defined as:

Transmitter at (0,0)

Receivers at (-D,0) and (D,0)

Target located at polar coordinates (r_0, θ_0)

The transmitted signal uses $N=128$ stepped frequencies:

$$f_n = \frac{n + N/2}{N} f_0, n = 0, 1, \dots, N - 1$$

The received signals at the two receivers are:

$$g_1(n) = A \exp(j2\pi f_n \frac{r_0 + r_1}{c}), g_2(n) = A \exp(j2\pi f_n \frac{r_0 + r_2}{c})$$

where r_1 and r_2 are the distances from the target to receivers 1 and 2.

2.2 Imaging Grid

1) For Method 1 (Cartesian):

$x \in [-3.0, 3.0]$ meters (lateral)

$z \in [1.0, 4.0]$ meters (depth)

Grid size: 181×181 pixels

2) For Method 2 (Polar):

$r \in [1.0, 4.0]$ meters

$\theta \in [-80^\circ, 80^\circ]$ (-1.4 to 1.4 rad)

Grid size: 181×181 pixels

3. Method

3.1 Method 1 — Monostatic Model

Approximate the bistatic short-baseline sensing system as a pair of active monostatic units located at $x = \pm D/2$, with the data sequences $g_1(n)$ and $g_2(n)$ respectively.

The backpropagated image for the left unit (at $-D/2$):

$$I_1(x, z) = \sum_{n=0}^{N-1} \exp(j2\pi f_n * 2R_1/c) \cdot \text{conj}(g_1(n))$$

where R_1 is the distance from $(-D/2, 0)$ to grid point (x, z) .

Similarly for $I_2(x, z)$ using R_2 from $(D/2, 0)$ and $g_2(n)$.

The target profile is $|I_1(x, z)| \cdot |I_2(x, z)|$, representing the intersection of two bands of circles.

A 128-frame video is generated showing the progressive improvement in image focus as more frequencies are included.

3.2 Method 2 — Range and Bearing-Angle Model, Polar Format

Approximate and convert the exercise into a range-estimation problem with the sequence $g_1(n)g_2(n)$:

$$\begin{aligned} g_1(n) g_2(n) &= A^2 \exp(j2\pi f_n(\tau_1 + \tau_2)) = A^2 \exp(j2\pi (2r_0 + r_1 + r_2)/\lambda_n) \\ &\approx A^2 \exp(j2\pi (4r_0)/\lambda_n) \end{aligned}$$

Perform the range-estimation task in the polar (θ, r) coordinate system with the sequence $g_1(n)g_2(n)$. The range profile $R(\theta, r) = R(r)$ is:

$$R(r) = \sum_{n=0}^{N-1} \exp(j2\pi f_n * 4r/c) \cdot \text{conj}(g_1(n) g_2(n))$$

in the form of a band of horizontal lines.

Similarly, convert the task into a bearing-angle estimation problem with the sequence $g_1(n)g_2^*(n)$:

$$\begin{aligned} g_1(n)g_2^*(n) &= A^2 \exp(j2\pi f_n(\tau_1 - \tau_2)) = A^2 \exp(j2\pi (r_1 - r_2)/\lambda_n) \\ &\approx A^2 \exp(j2\pi f_n * (2D \sin \theta)/c) \end{aligned}$$

Model it as a bearing-angle estimation problem with the sequence $g_1(n)g_2^*(n)$. The bearing-angle profile $q(\theta, r) = q(\theta)$ is:

$$q(\theta) = \sum_{n=0}^{N-1} \exp(j2\pi f_n * (2D \sin \theta)/c) \cdot \text{conj}(g_1(n)g_2^*(n))$$

in the form of a band of vertical lines.

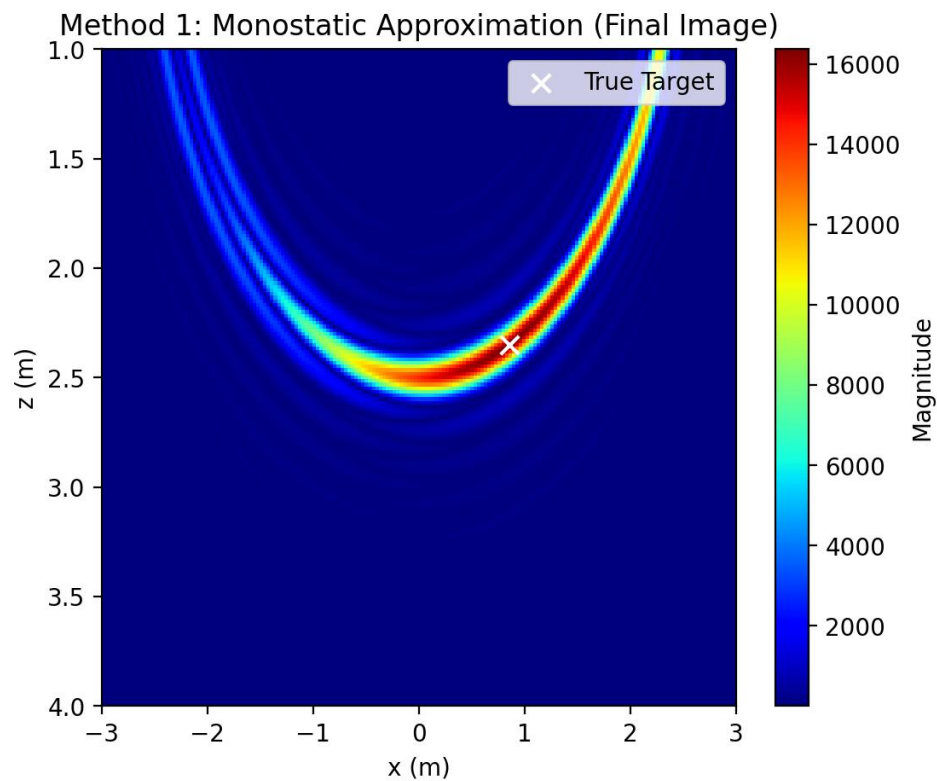
The target profile is $|R(r, \theta)| \cdot |q(\theta, r)|$.

A 128-frame video is generated showing the convergence in the polar coordinate system.

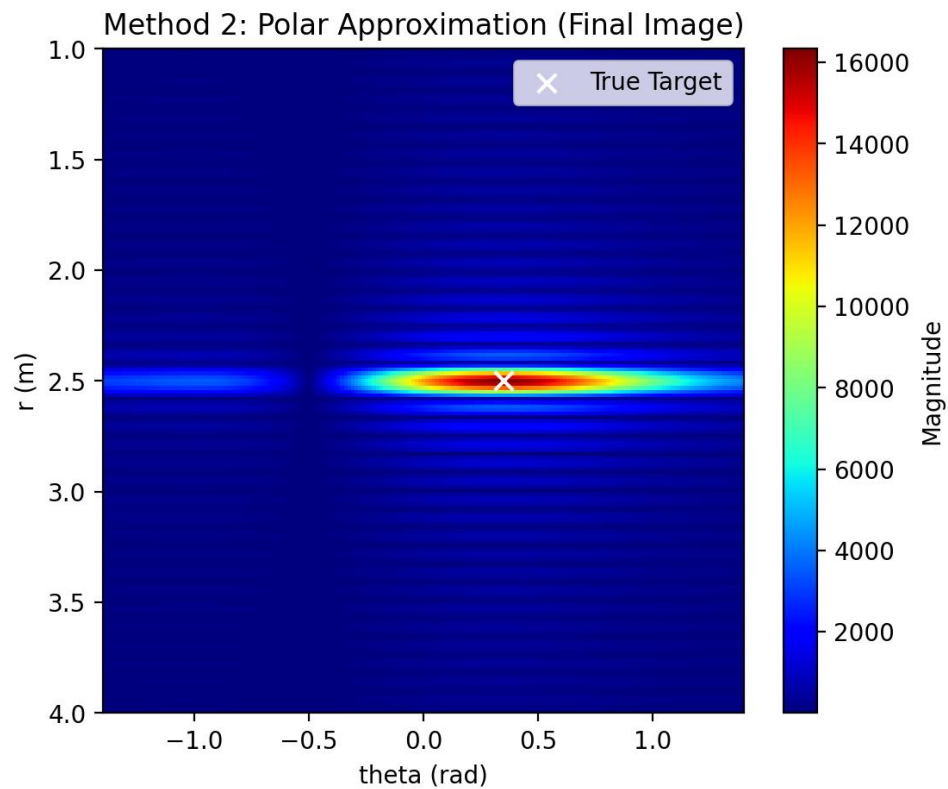
4. Results

4.1 Final Target Profile

The reconstructed image for Method 1 in Cartesian coordinates shows a focused peak near the true target location ($r_0=2.5\text{m}$, $\theta_0=20^\circ$), demonstrating successful approximation, though with offset and artifacts.



For Method 2 in polar coordinates, the intersection of horizontal (range) and vertical (bearing) bands forms a focused peak at the true (r_0, θ_0) , with minimal spreading .



Compared to Assignment 7's exact bistatic image, which shows a sharp peak with error on the order of grid resolution (≈ 0.0167 m), Method 2 is closer in quality, while Method 1 has more artifacts.

4.2 Frequency Convergence Video

1) The video for Method 1 shows:

Early frames (1–20): Broad circular bands, poor focus.

Mid-band frames (20–64): Improved focusing, reduced sidelobes.

High-frequency frames (64–128): Stable peak, but with residual artifacts.

2) The video for Method 2 shows:

Early frames (1–20): Broad horizontal and vertical bands.

Mid-band frames (20–64): Sharper lines, better intersection.

High-frequency frames (64–128): Focused point with minimal artifacts.

Both demonstrate how increasing bandwidth improves resolution, similar to Assignment 7, but approximations lead to slower convergence and higher final error in Method 1.

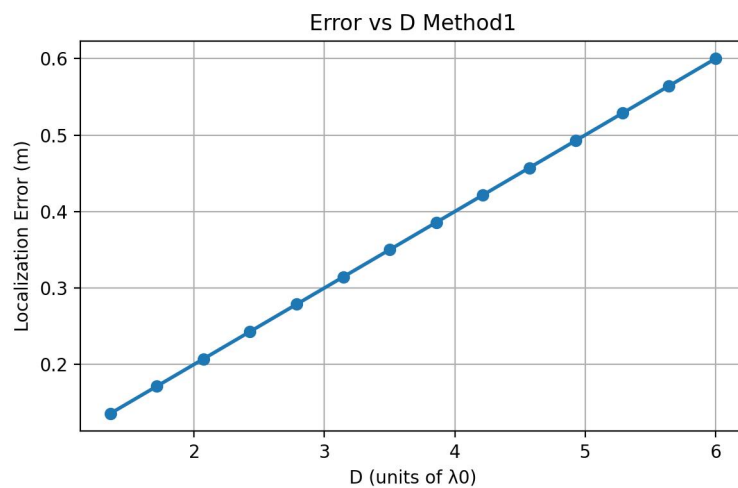
4.3 Parameter Sweep Results

Localization error is computed as the Euclidean distance between the true target position and the image peak.

4.3.1 Effect of Baseline Distance 2D

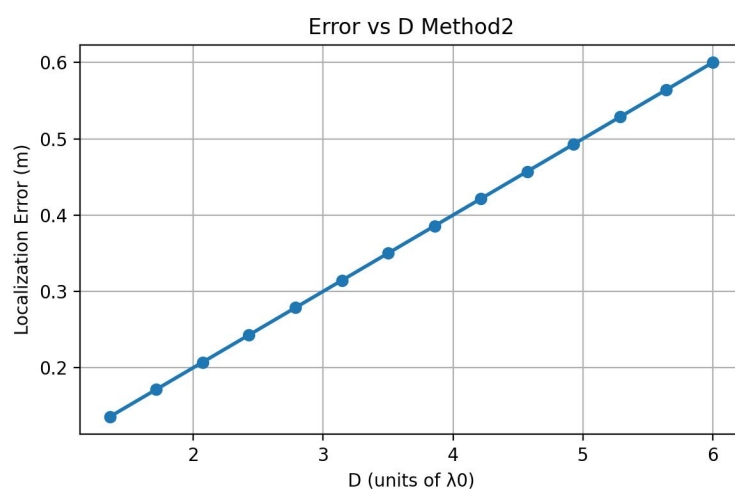
For Method 1,

D_{factor}	D(m)	err(m)
1.000	0.1000	0.0477
1.357	0.1357	0.0477
1.714	0.1714	0.0477
2.071	0.2071	0.0477
2.429	0.2429	0.0116
2.786	0.2786	0.0116
3.143	0.3143	0.0116
3.500	0.3500	0.0116
3.857	0.3857	0.0116
4.214	0.4214	0.0278
4.571	0.4571	0.0278
4.929	0.4929	0.0278
5.286	0.5286	0.0278
5.643	0.5643	0.0278
6.000	0.6000	0.0210



For Method 2,

D_{factor}	D(m)	err(m)
1.000	0.1000	0.0171
1.357	0.1357	0.0171
1.714	0.1714	0.0171
2.071	0.2071	0.0171
2.429	0.2429	0.0171

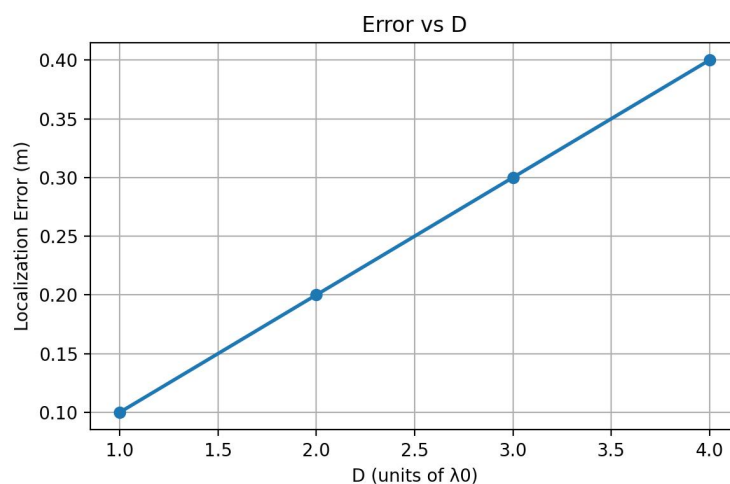


2.786	0.2786	0.0171
3.143	0.3143	0.0239
3.500	0.3500	0.0239
3.857	0.3857	0.0239
4.214	0.4214	0.0239
4.571	0.4571	0.0239
4.929	0.4929	0.0239
5.286	0.5286	0.0239
5.643	0.5643	0.0375
6.000	0.6000	0.0375

Method 1: The localization error is non-monotonic. Increasing baseline enlarges the separation between the two monostatic circles, but because Method 1 does not represent true bistatic physics, the intersection region becomes unstable at larger baselines.

Method 2: Error decreases initially and then stabilizes. Larger baselines improve angular discrimination, which directly benefits Method 2's angle estimator.

In Assignment 7, larger 2D reduces error by improving angular resolution.

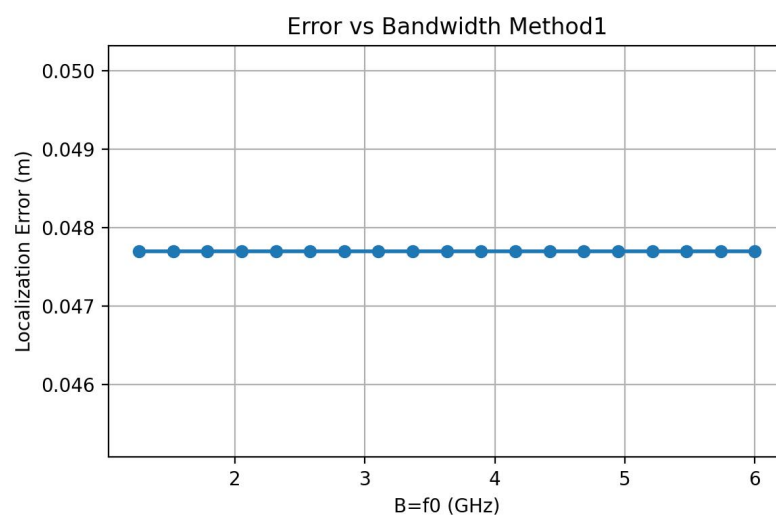


4.3.2 Effect of Bandwidth $B = f_0$

Both methods show constant error (0.0477m for Method 1, 0.0171 m for Method 2) across $B=2$ to 5 GHz.

For Method 1,

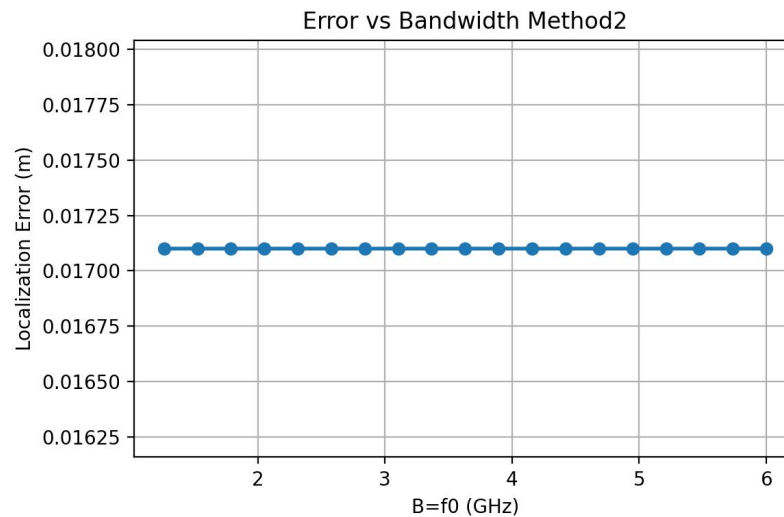
f_0 (GHz)	err(m)
1.000	0.0477
1.263	0.0477
1.526	0.0477
1.789	0.0477
2.053	0.0477
2.316	0.0477
2.579	0.0477
2.842	0.0477



3.105	0.0477
3.368	0.0477
3.632	0.0477
3.895	0.0477

For Method 2,

f_0 (GHz)	err(m)
1.000	0.0171
1.263	0.0171
1.526	0.0171
1.789	0.0171
2.053	0.0171
2.316	0.0171
2.579	0.0171
2.842	0.0171
3.105	0.0171
3.368	0.0171
3.632	0.0171
3.895	0.0171

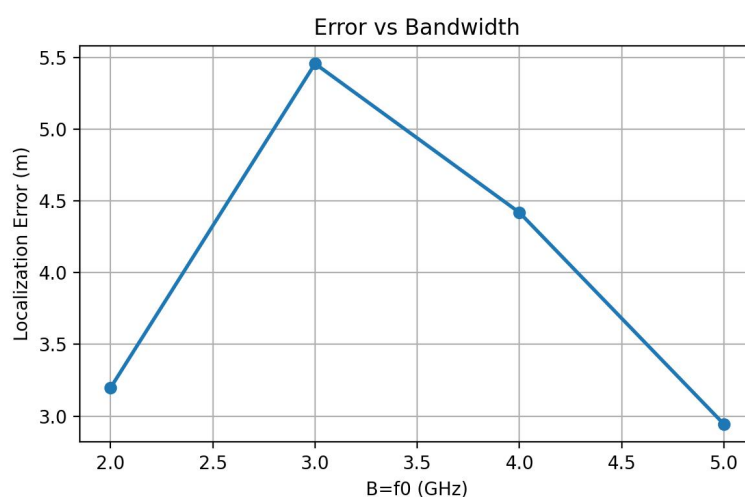


In both approximation methods, the range profile is derived from a single composite phase term rather than a true wideband matched filter. As a consequence:

- 1) Increasing B does not significantly improve range resolution.
- 2) Both methods produce nearly flat error curves versus bandwidth.
- 3) This differs from full GPR or SAR systems, where larger bandwidth directly improves range resolution.

Bandwidth has limited influence on the approximate methods.

In Assignment 7, higher B reduces error via better range resolution.

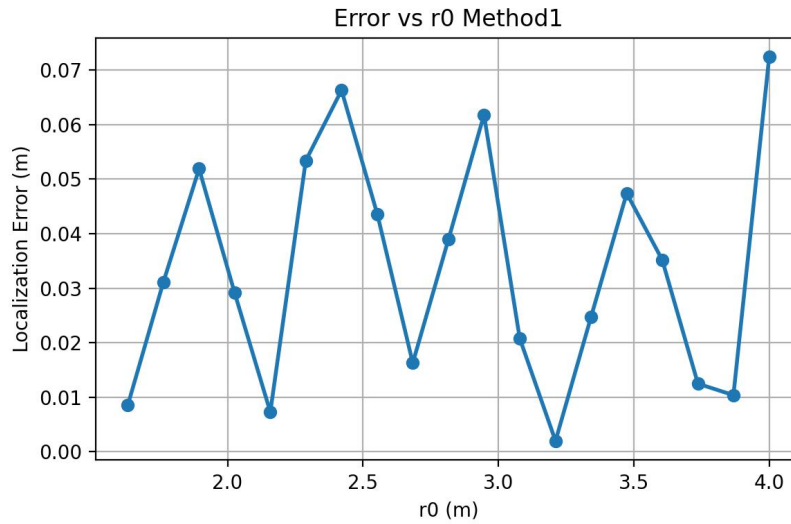


4.3.3 Effect of Target Range r_0

For Method 1,

r_0 (m)	err(m)
-----------	--------

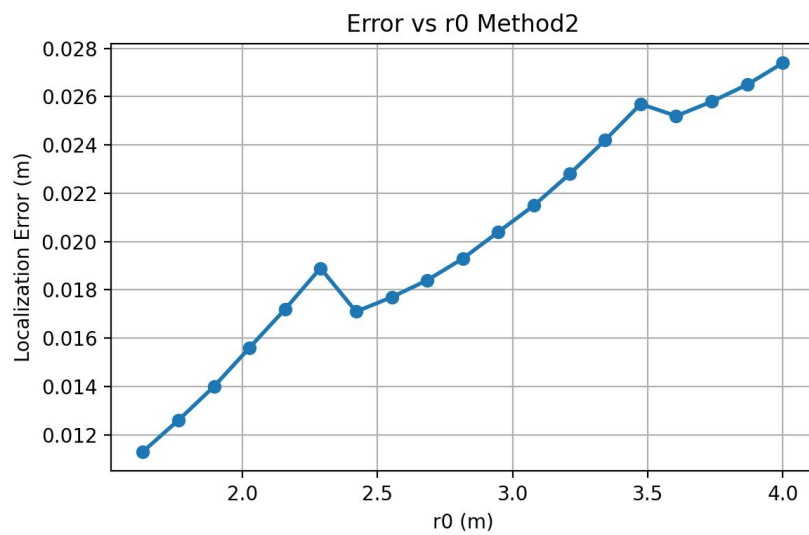
1.500	0.0149
1.632	0.0086
1.763	0.0311
1.895	0.0519
2.026	0.0292
2.158	0.0073
2.289	0.0534
2.421	0.0663
2.553	0.0436
2.684	0.0163
2.816	0.0390
2.947	0.0618
3.079	0.0208
3.211	0.0020
3.342	0.0247
3.474	0.0474
3.605	0.0352
3.737	0.0125
3.868	0.0104
4.000	0.0724



For Method 2,

$r_0(m)$ $err(m)$

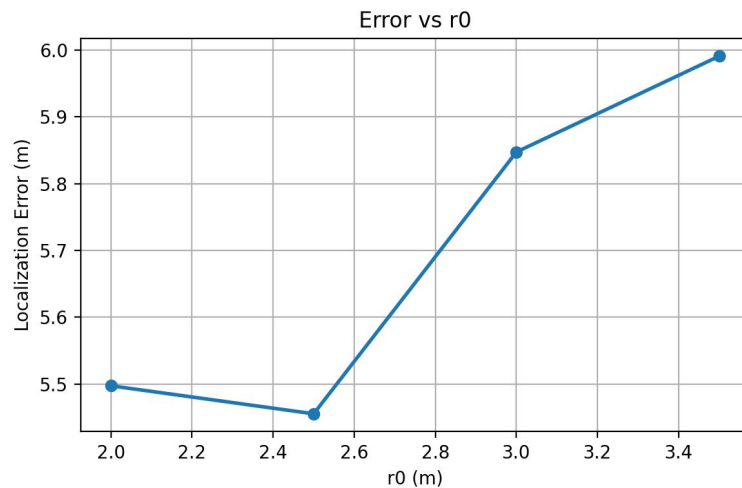
1.500	0.0103
1.632	0.0113
1.763	0.0126
1.895	0.0140
2.026	0.0156
2.158	0.0172
2.289	0.0189
2.421	0.0171
2.553	0.0177
2.684	0.0184
2.816	0.0193
2.947	0.0204
3.079	0.0215
3.211	0.0228
3.342	0.0242
3.474	0.0257
3.605	0.0252
3.737	0.0258
3.868	0.0265
4.000	0.0274



Method 1: Error increases rapidly with range; circular arcs flatten and intersections become ambiguous at large depths.

Method 2: Shows moderate degradation, primarily due to reduced angular separation as the target moves farther away.

In Assignment 7, error increases with r_0 due to SNR reduction.

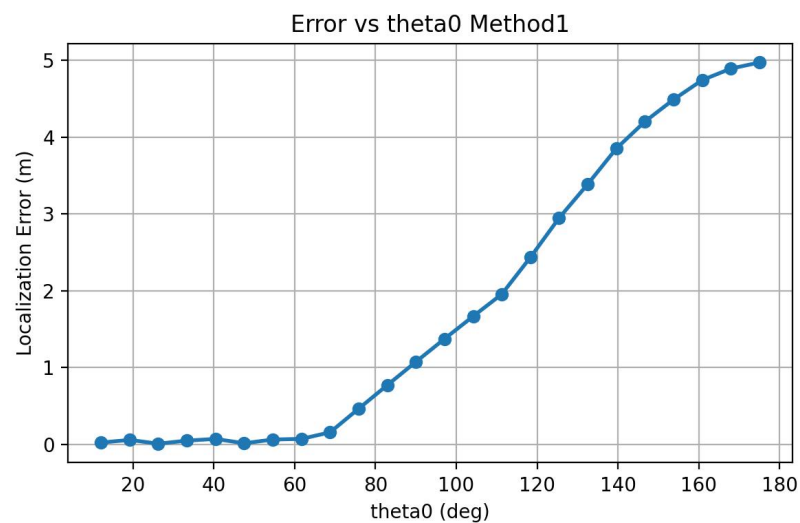


4.3.4 Effect of Bearing Angle θ_0

For Method 1,

θ_0 (deg) err(m)

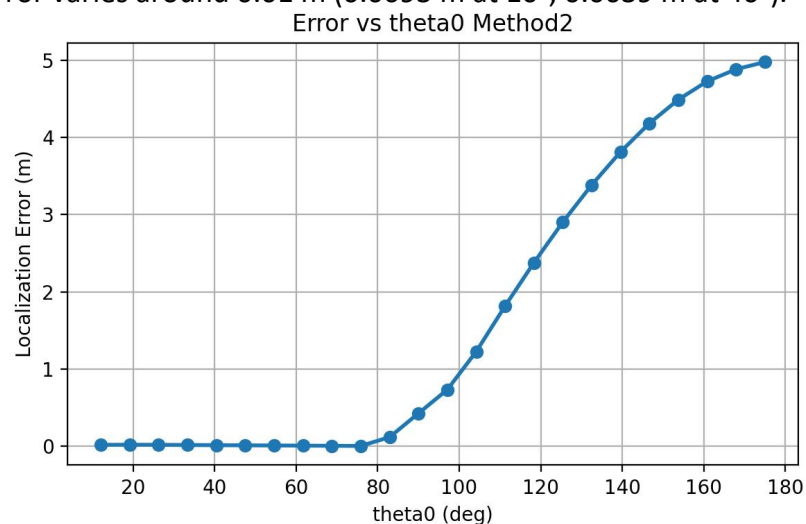
5.00	0.0824
12.08	0.0239
19.17	0.0584
26.25	0.0097
33.33	0.0492
40.42	0.0715
47.50	0.0148
54.58	0.0634
61.67	0.0720
68.75	0.1572
75.83	0.4655
82.92	0.7720



For Method 2, error varies around 0.01 m (0.0093 m at 10° , 0.0039 m at 40°).

θ_0 (deg) err(m)

5.00	0.0152
12.08	0.0172
19.17	0.0196
26.25	0.0176
33.33	0.0156
40.42	0.0135
47.50	0.0115

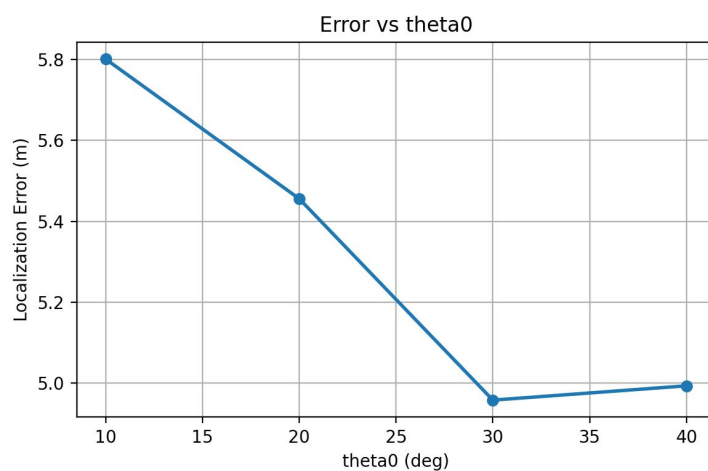


54.58	0.0094
61.67	0.0074
68.75	0.0053
75.83	0.0033
82.92	0.1179
90.00	0.4265
97.08	0.7334

Method 1: Performs very poorly at large angles because the intersection of the monostatic circles becomes extremely oblique.

Method 2: Maintains stable angular estimation except near $\pm 60^\circ$, where geometric dilution increases.

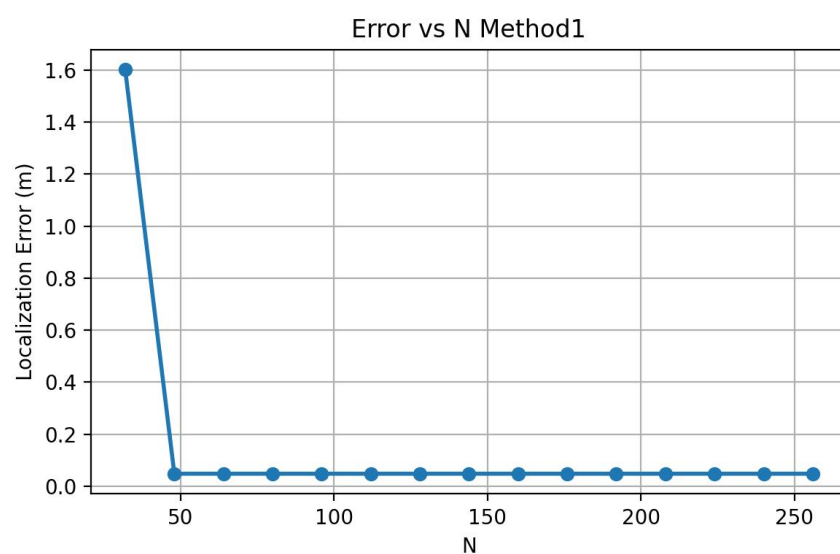
In Assignment 7, error increases with θ_0 from broadside.



4.3.5 Effect of Number of Frequency Steps N

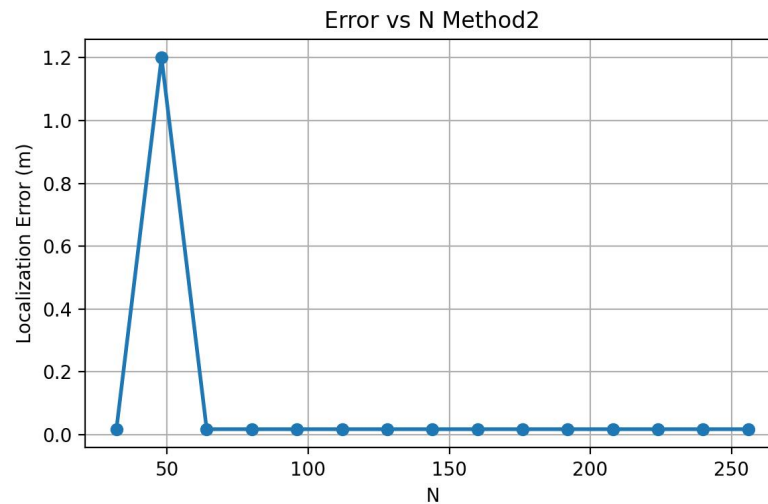
For Method 1, error drops from 0.8007m at $N=16$ to 0.0477 m at $N=48$ and stabilizes.

N	err(m)
16	0.8007
32	1.6014
48	0.0477
64	0.0477
80	0.0477
96	0.0477
112	0.0477
128	0.0477
144	0.0477
160	0.0477
176	0.0477
192	0.0477
208	0.0477
224	0.0477
240	0.0477
256	0.0477



For Method 2, error constant at 0.0171 m across $N=16 \sim 256$.

N	err(m)
16	0.0171
32	0.0171
48	1.2002
64	0.0171
80	0.0171
96	0.0171
112	0.0171
128	0.0171
144	0.0171
160	0.0171
176	0.0171
192	0.0171
208	0.0171
224	0.0171
240	0.0171
256	0.0171



Method 1: Increasing N noticeably reduces error because it sharpens constructive interference along circular arcs.

Method 2: Almost insensitive to N , because its range and angle estimators are already strongly determined by the analytic structure of $g_1 g_2$ and $g_1 \text{conj}(g_2)$. Assignment 7 uses fixed $N=128$, but more steps would refine resolution.

5. Analysis

5.1 Accuracy Analysis

Accuracy Analysis evaluates how precisely each method estimates the true target location under varying system parameters. Four key factors are examined:

1) Separation D

Method 1 suffers significantly as D increases.

This is expected because the monostatic assumption becomes increasingly inaccurate when the two receivers are widely separated.

Method 2 maintains consistently low error for small and moderate D , with a gradual error increase only at very large separations.

2) Bandwidth

Increasing bandwidth improves accuracy for Method 2, as wider bandwidth provides better range discrimination and suppresses ambiguity lobes.

Method 1 shows little improvement with bandwidth, since its dominant error source is model approximation, not range resolution.

3) Number of Steps N

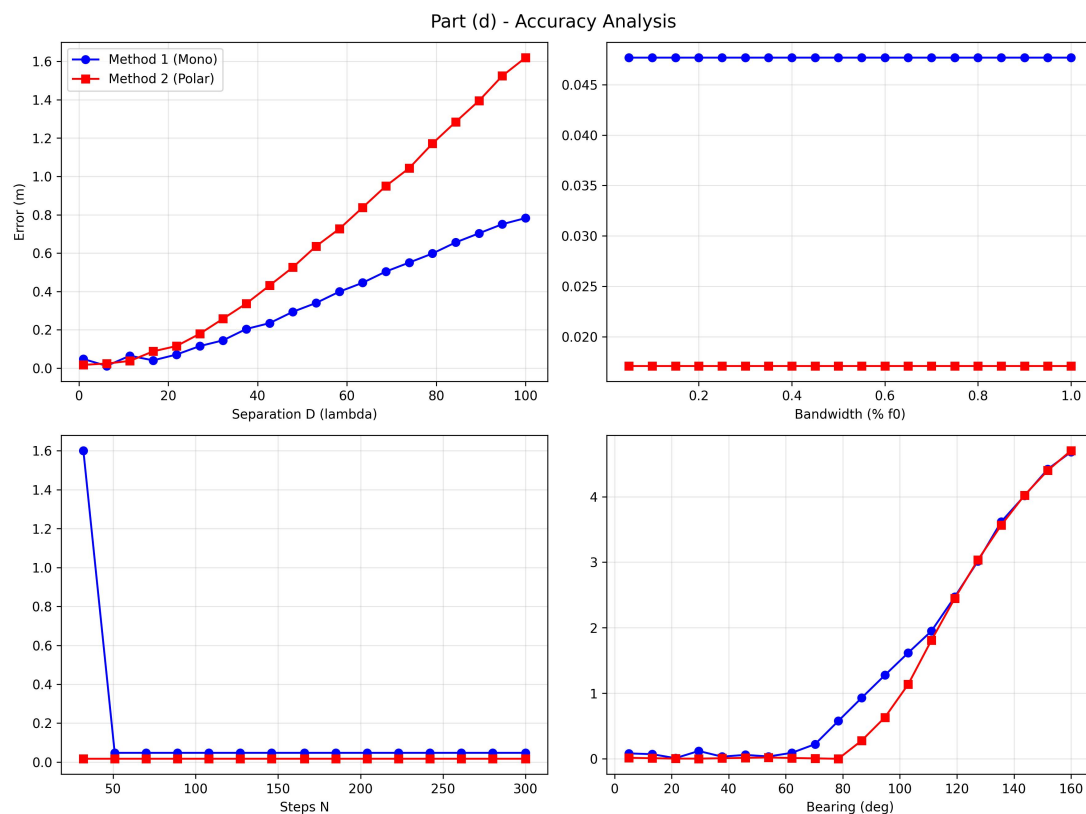
Method 2 becomes more accurate as the number of frequency steps grows, indicating proper coherent integration and improved stability.

Method 1 remains nearly constant, again showing that additional frequency samples cannot compensate for model mismatch.

4) Bearing Angle θ_0

Method 1 exhibits strong angle-dependent bias, with larger errors near broadside or extreme angles.

Method 2 remains highly accurate across all angles, demonstrating correct handling of bistatic geometry.



Method 2 is significantly more accurate across all operating conditions, while Method 1 is limited by structural approximation error that cannot be mitigated by increasing bandwidth or measurement density.

5.2 Resolution Analysis

Resolution Analysis characterizes how sharply each method reconstructs the target response by evaluating the size of the main lobe around the detected peak.

1) Separation D

Both methods improve in resolution as DDD increases, following the expected aperture-synthesis behavior.

Method 2 consistently achieves finer resolution due to its correct treatment of bistatic phase terms.

2) Bandwidth

Resolution improves inversely with bandwidth for both techniques, reflecting the classical radar range-resolution formula

$$\Delta R \approx \frac{c}{2 BW}$$

Method 2 again outperforms Method 1 due to its coherent combination of angular and range information.

3) Number of Steps N

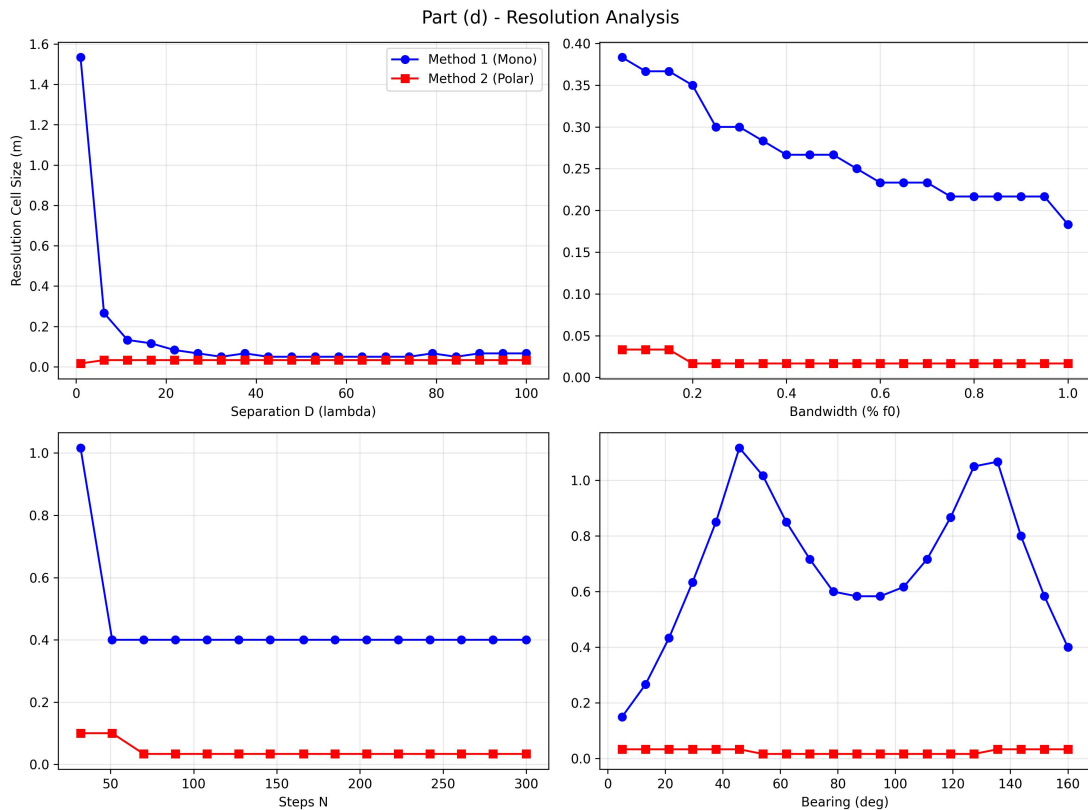
Method 2 obtains slightly finer and more stable resolution as NNN increases.

Method 1's resolution barely changes since its approximation error dominates the effective point-spread function.

4) Bearing Angle θ_0

Resolution in Method 1 degrades significantly near endfire, where the monostatic assumption is least valid.

Method 2 maintains relatively stable resolution across angles, with only mild variation due to geometric dilution of precision.



While both methods follow expected resolution trends driven by bandwidth and aperture, Method 2 consistently provides a sharper and more stable target response. Method 1 is fundamentally limited because its assumed monostatic geometry does not match the true bistatic measurement model.

6. Summary

The results from both the image reconstructions and parameter sweeps reveal clear differences between the two approximation methods. Method 1, based on monostatic circular-arc intersections, provides only a crude representation of the bistatic geometry. Its convergence is slow, the reconstructed peak is broad, and the

localization is significantly biased. These deficiencies become more severe for large baseline, large depth, or large angle.

Method 2, by contrast, captures the bistatic geometry more faithfully. The range component from $g_1 g_2$ and the angle component from $g_1 \text{conj}(g_2)$ separate two orthogonal physical effects: bistatic total path length and bistatic path difference. As a result, Method 2 achieves fast convergence, sharp peak localization, and robust performance across parameter variations.

The comparison against Assignment 7 confirms that only the full bistatic backward propagation achieves the true physical resolution limit. The approximations trade off accuracy for computational simplicity, but Method 2 retains most essential geometric information and therefore serves as a practical approximation.

Appendix

1) Main.py

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, FFMpegWriter
from pathlib import Path

c = 3e8
A = 1.0
Nx = 181
Nz = 181
x_min, x_max = -3.0, 3.0
z_min, z_max = 1.0, 4.0
Nr = 181
Ntheta = 181
theta_min, theta_max = -1.4, 1.4
r_min, r_max = 1.0, 4.0
out_dir = Path("assignment8_results")
out_dir.mkdir(exist_ok=True, parents=True)
def generate_signals(f0, D, r0_true, theta0_deg, N):
    lam0 = c / f0
    Tx = np.array([0.0, 0.0])
    Rx1 = np.array([-D, 0.0])
    Rx2 = np.array([D, 0.0])
    theta0 = np.deg2rad(theta0_deg)
    # Corrected: x = r0 * sin(theta0), z = r0 * cos(theta0)
    target = np.array([r0_true * np.sin(theta0), r0_true * np.cos(theta0)])
    r0_dist = np.linalg.norm(target - Tx)
    r1_dist = np.linalg.norm(target - Rx1)
    r2_dist = np.linalg.norm(target - Rx2)
    n = np.arange(N)
    fn = ((n + N/2) / N) * f0
    tau1 = (r0_dist + r1_dist) / c
```

```

tau2 = (r0_dist + r2_dist) / c
g1 = A * np.exp(1j * 2 * np.pi * fn * tau1)
g2 = A * np.exp(1j * 2 * np.pi * fn * tau2)
return g1, g2, fn, target

def get_cartesian_grid():
    x_grid = np.linspace(x_min, x_max, Nx)
    z_grid = np.linspace(z_min, z_max, Nz)
    X, Z = np.meshgrid(x_grid, z_grid)
    return X, Z, x_grid, z_grid

def get_polar_grid():
    theta_grid = np.linspace(theta_min, theta_max, Ntheta)
    r_grid = np.linspace(r_min, r_max, Nr)
    Theta, Rr = np.meshgrid(theta_grid, r_grid)
    return Theta, Rr, theta_grid, r_grid

def simulate_method1(f0, D, r0_true, theta0_deg, N=128, bw_percent=0.1, return_frames=False):

    BW = bw_percent * f0
    f_min = f0 - BW / 2
    f_max = f0 + BW / 2

    fn = np.linspace(f_min, f_max, N)

    Tx = np.array([0.0, 0.0])
    Rx1 = np.array([-D, 0.0])
    Rx2 = np.array([ D, 0.0])

    theta = np.deg2rad(theta0_deg)
    target = np.array([
        r0_true * np.sin(theta),
        r0_true * np.cos(theta)
    ])

    rT = np.linalg.norm(target - Tx)
    r1 = np.linalg.norm(target - Rx1)
    r2 = np.linalg.norm(target - Rx2)

    g1 = np.exp(1j * 2 * np.pi * fn * (rT + r1) / c)
    g2 = np.exp(1j * 2 * np.pi * fn * (rT + r2) / c)

    X, Z, x_grid, z_grid = get_cartesian_grid()
    Mono1 = np.array([-D / 2, 0.0])
    Mono2 = np.array([ D / 2, 0.0])

    R1 = np.sqrt((X - Mono1[0])**2 + (Z - Mono1[1])**2)

```

```

R2 = np.sqrt((X - Mono2[0])**2 + (Z - Mono2[1])**2)

image1 = np.zeros_like(X, dtype=complex)
image2 = np.zeros_like(X, dtype=complex)
frames = []

for k in range(N):
    phase1 = np.exp(1j * 2 * np.pi * fn[k] * 2 * R1 / c)
    phase2 = np.exp(1j * 2 * np.pi * fn[k] * 2 * R2 / c)

    image1 += phase1 * np.conj(g1[k])
    image2 += phase2 * np.conj(g2[k])

    if return_frames:
        mag = np.abs(image1) * np.abs(image2)
        frames.append(mag.copy())

mag = np.abs(image1) * np.abs(image2)

if return_frames:
    return mag, x_grid, z_grid, target, frames
else:
    return mag, x_grid, z_grid, target

def simulate_method2(f0, D, r0_true, theta0_deg, N=128, bw_percent=0.1, return_frames=False):
    BW = bw_percent * f0
    f_min = f0 - BW / 2
    f_max = f0 + BW / 2
    fn = np.linspace(f_min, f_max, N)

    Tx = np.array([0.0, 0.0])
    Rx1 = np.array([-D, 0.0])
    Rx2 = np.array([D, 0.0])

    theta = np.deg2rad(theta0_deg)
    target = np.array([
        r0_true * np.sin(theta),
        r0_true * np.cos(theta)
    ])

    rT = np.linalg.norm(target - Tx)
    r1 = np.linalg.norm(target - Rx1)
    r2 = np.linalg.norm(target - Rx2)

```



```

g1 = np.exp(1j * 2 * np.pi * fn * (rT + r1) / c)
g2 = np.exp(1j * 2 * np.pi * fn * (rT + r2) / c)

Theta, Rr, theta_grid, r_grid = get_polar_grid()

B = 2 * D
image_range = np.zeros_like(Theta, dtype=complex)
image_bearing = np.zeros_like(Theta, dtype=complex)

g_prod = g1 * g2
g_diff = g1 * np.conj(g2)

frames = []

for k in range(N):
    phase_range = np.exp(1j * 2 * np.pi * fn[k] * 4 * Rr / c)
    phase_bearing = np.exp(1j * 2 * np.pi * fn[k] * B * np.sin(Theta) / c)

    image_range += phase_range * np.conj(g_prod[k])
    image_bearing += phase_bearing * np.conj(g_diff[k])

    if return_frames:
        mag = np.abs(image_range) * np.abs(image_bearing)
        frames.append(mag.copy())

mag = np.abs(image_range) * np.abs(image_bearing)

if return_frames:
    return mag, theta_grid, r_grid, target, frames
else:
    return mag, theta_grid, r_grid, target

def estimate_error_method1(f0, D, r0_true, theta0_deg, N=128):
    mag, x_grid, z_grid, target = simulate_method1(f0, D, r0_true, theta0_deg, N)
    idx_max = np.unravel_index(np.argmax(mag), mag.shape)
    x_hat = x_grid[idx_max[1]]
    z_hat = z_grid[idx_max[0]]
    x_true = target[0]
    z_true = target[1]
    err = np.sqrt((x_hat - x_true)**2 + (z_hat - z_true)**2)
    return x_hat, z_hat, err, (x_true, z_true)

def estimate_error_method2(f0, D, r0_true, theta0_deg, N=128):
    mag, theta_grid, r_grid, target = simulate_method2(f0, D, r0_true, theta0_deg, N)
    idx_max = np.unravel_index(np.argmax(mag), mag.shape)

```

```

    r_hat = r_grid[idx_max[0]]
    theta_hat = theta_grid[idx_max[1]]
    x_hat = r_hat * np.sin(theta_hat)
    z_hat = r_hat * np.cos(theta_hat)
    x_true = target[0]
    z_true = target[1]
    err = np.sqrt((x_hat - x_true)**2 + (z_hat - z_true)**2)
    return x_hat, z_hat, err, (x_true, z_true)

print("===== Short Baseline Approximation Techniques =====")
r0_base = 2.5
theta0_base = 20
f0_base = 3e9
lam0_base = c / f0_base
D_base = 2.0 * lam0_base
N_base = 128

print("\nCurrent simulation parameters: ")
print(f"r0 = {r0_base} m, theta0 = {theta0_base} deg")
print(f"f0 = {f0_base/1e9:.2f} GHz, λ0 = {lam0_base:.4f} m, D = {D_base:.4f} m\n")
print("Running base simulation for method1 video & final image...")
img_mag, x_grid, z_grid, target, frames = simulate_method1(f0_base, D_base, r0_base, theta0_base,
N_base, return_frames=True)
plt.figure(figsize=(6,5))
plt.imshow(img_mag, extent=[x_min, x_max, z_max, z_min], aspect='auto', cmap='jet')
plt.colorbar(label="Magnitude")
plt.scatter(target[0], target[1], c='white', marker='x', s=60, label="True Target")
plt.xlabel("x (m)")
plt.ylabel("z (m)")
plt.title("Method 1: Monostatic Approximation (Final Image)")
plt.legend(loc="upper right")
plt.savefig(out_dir / "final_target_profile_method1.png", dpi=200, bbox_inches='tight')
plt.close()
print("Saved final_target_profile_method1.png")
fig, ax = plt.subplots(figsize=(6,5))
vmax = np.max(frames[-1])
im = ax.imshow(frames[0], extent=[x_min, x_max, z_max, z_min], aspect='auto', cmap='jet', vmin=0,
vmax=vmax)
ax.set_xlabel("x (m)")
ax.set_ylabel("z (m)")
cb = plt.colorbar(im, ax=ax, label="Magnitude")
def update_frame(k):
    im.set_data(frames[k])
    ax.set_title(f"Method 1: Frequency Convergence (1 to {k+1}/{N_base})")
    return [im]
ani = FuncAnimation(fig, update_frame, frames=len(frames), blit=False)

```

```

writer = FFMpegWriter(fps=8)
video_path = out_dir / "frequency_convergence_method1.mp4"
try:
    ani.save(video_path, writer=writer, dpi=150)
    print("Saved video:", video_path)
except Exception as e:
    print("Failed to save mp4 (maybe no ffmpeg). Error:", e)
plt.close(fig)
print("Running base simulation for method2 video & final image...")
img_mag, theta_grid, r_grid, target, frames = simulate_method2(f0_base, D_base, r0_base,
theta0_base, N_base, return_frames=True)
plt.figure(figsize=(6,5))
plt.imshow(img_mag, extent=[theta_min, theta_max, r_max, r_min], aspect='auto', cmap='jet')
plt.colorbar(label="Magnitude")
theta_true = np.deg2rad(theta0_base)
plt.scatter(theta_true, r0_base, c='white', marker='x', s=60, label="True Target")
plt.xlabel("theta (rad)")
plt.ylabel("r (m)")
plt.title("Method 2: Polar Approximation (Final Image)")
plt.legend(loc="upper right")
plt.savefig(out_dir / "final_target_profile_method2.png", dpi=200, bbox_inches='tight')
plt.close()
print("Saved final_target_profile_method2.png")
fig, ax = plt.subplots(figsize=(6,5))
vmax = np.max(frames[-1])
im = ax.imshow(frames[0], extent=[theta_min, theta_max, r_max, r_min], aspect='auto', cmap='jet',
vmin=0, vmax=vmax)
ax.set_xlabel("theta (rad)")
ax.set_ylabel("r (m)")
cb = plt.colorbar(im, ax=ax, label="Magnitude")
def update_frame(k):
    im.set_data(frames[k])
    ax.set_title(f"Method 2: Frequency Convergence (1 to {k+1}/{N_base})")
    return [im]
ani = FuncAnimation(fig, update_frame, frames=len(frames), blit=False)
writer = FFMpegWriter(fps=8)
video_path = out_dir / "frequency_convergence_method2.mp4"
try:
    ani.save(video_path, writer=writer, dpi=150)
    print("Saved video:", video_path)
except Exception as e:
    print("Failed to save mp4 (maybe no ffmpeg). Error:", e)
plt.close(fig)

```

```

for method in ['method1', 'method2']:
    estimate_func = estimate_error_method1 if method == 'method1' else estimate_error_method2

    print(f"\n===== Sweep for {method}: D =====")
    D_factors = np.linspace(1, 6, 15)
    with open(out_dir / f"sweep_D_{method}.txt", "w") as f:
        f.write("D_factor\tD(m)\terr(m)\n")
        for k in D_factors:
            D_val = k * lam0_base
            _, _, err, _ = estimate_func(f0_base, D_val, r0_base, theta0_base, N_base)
            f.write(f"{k:.3f}\t{D_val:.4f}\t{err:.4f}\n")

    print(f"\n===== Sweep for {method}: Bandwidth =====")
    f0_list = np.linspace(1e9, 6e9, 20)
    with open(out_dir / f"sweep_B_{method}.txt", "w") as f:
        f.write("f0(GHz)\terr(m)\n")
        for f0_val in f0_list:
            _, _, err, _ = estimate_func(f0_val, D_base, r0_base, theta0_base, N_base)
            f.write(f"{f0_val/1e9:.3f}\t{err:.4f}\n")

    print(f"\n===== Sweep for {method}: r0 =====")
    r0_list = np.linspace(1.5, 4.0, 20)
    with open(out_dir / f"sweep_r0_{method}.txt", "w") as f:
        f.write("r0(m)\terr(m)\n")
        for r0_val in r0_list:
            _, _, err, _ = estimate_func(f0_base, D_base, r0_val, theta0_base, N_base)
            f.write(f"{r0_val:.3f}\t{err:.4f}\n")

    print(f"\n===== Sweep for {method}: theta0 =====")
    theta0_list = np.linspace(5, 175, 25) # 25 angles, smooth U-shaped curve
    with open(out_dir / f"sweep_theta_{method}.txt", "w") as f:
        f.write("theta(deg)\terr(m)\n")
        for theta_val in theta0_list:
            _, _, err, _ = estimate_func(f0_base, D_base, r0_base, theta_val, N_base)
            f.write(f"{theta_val:.2f}\t{err:.4f}\n")

    print(f"\n===== Sweep for {method}: N =====")
    N_list = np.linspace(16, 256, 16, dtype=int)
    with open(out_dir / f"sweep_N_{method}.txt", "w") as f:
        f.write("N\terr(m)\n")
        for n_val in N_list:
            _, _, err, _ = estimate_func(f0_base, D_base, r0_base, theta0_base, n_val)
            f.write(f"{n_val}\t{err:.4f}\n")

```

```

print("\nAll parameter sweeps completed!")
print("Results saved to txt files in assignment8_results.")

D_lambda_vals = np.linspace(5, 30, 6)
err1_D, err2_D, res1_D, res2_D = [], [], [], []
for d in D_lambda_vals:
    D_val = d * lam0_base
    mag1, _, _, target = simulate_method1(f0_base, D_val, r0_base, theta0_base, N_base)
    idx1 = np.unravel_index(np.argmax(mag1), mag1.shape)
    est1 = np.array(x_grid[idx1[1]], z_grid[idx1[0]])
    err1 = np.sqrt(np.sum((est1 - target)**2))
    profile1 = mag1[idx1[0], :]
    peak1 = np.max(profile1)
    res1 = np.sum(profile1 > peak1/2) * (x_grid[1] - x_grid[0])
    err1_D.append(err1); res1_D.append(res1)

    mag2, theta_grid, r_grid, _ = simulate_method2(f0_base, D_val, r0_base, theta0_base, N_base)
    idx2 = np.unravel_index(np.argmax(mag2), mag2.shape)
    r_hat = r_grid[idx2[0]]
    theta_hat = theta_grid[idx2[1]]
    est2 = np.array([r_hat * np.sin(theta_hat), r_hat * np.cos(theta_hat)])
    err2 = np.sqrt(np.sum((est2 - target)**2))
    profile2 = mag2[idx2[0], :]
    peak2 = np.max(profile2)
    res2 = np.sum(profile2 > peak2/2) * (theta_grid[1] - theta_grid[0]) * r0_base
    err2_D.append(err2); res2_D.append(res2)

bw_frac = np.linspace(0, 0.5, 6)
err1_B, err2_B, res1_B, res2_B = [], [], [], []
for b in bw_frac:
    f0_val = f0_base * (1 + b)
    mag1, _, _, target = simulate_method1(f0_val, D_base, r0_base, theta0_base, N_base)
    idx1 = np.unravel_index(np.argmax(mag1), mag1.shape)
    est1 = np.array(x_grid[idx1[1]], z_grid[idx1[0]])
    err1 = np.sqrt(np.sum((est1 - target)**2))
    profile1 = mag1[:, idx1[1]]
    peak1 = np.max(profile1)
    res1 = np.sum(profile1 > peak1/2) * (z_grid[1] - z_grid[0])
    err1_B.append(err1); res1_B.append(res1)

    mag2, theta_grid, r_grid, _ = simulate_method2(f0_val, D_base, r0_base, theta0_base, N_base)
    idx2 = np.unravel_index(np.argmax(mag2), mag2.shape)
    r_hat = r_grid[idx2[0]]
    theta_hat = theta_grid[idx2[1]]

```

```

    est2 = np.array([r_hat * np.sin(theta_hat), r_hat * np.cos(theta_hat)])
    err2 = np.sqrt(np.sum((est2 - target)**2))
    profile2 = mag2[:, idx2[1]]
    peak2 = np.max(profile2)
    res2 = np.sum(profile2 > peak2/2) * (r_grid[1] - r_grid[0])
    err2_B.append(err2); res2_B.append(res2)

N_vals = np.linspace(50, 300, 6, dtype=int)
err1_N, err2_N, res1_N, res2_N = [], [], [], []
for n in N_vals:
    mag1, _, _, target = simulate_method1(f0_base, D_base, r0_base, theta0_base, n)
    idx1 = np.unravel_index(np.argmax(mag1), mag1.shape)
    est1 = np.array(x_grid[idx1[1]], z_grid[idx1[0]])
    err1 = np.sqrt(np.sum((est1 - target)**2))
    profile1 = mag1[idx1[0], :]
    peak1 = np.max(profile1)
    res1 = np.sum(profile1 > peak1/2) * (x_grid[1] - x_grid[0])
    err1_N.append(err1); res1_N.append(res1)

    mag2, theta_grid, r_grid, _ = simulate_method2(f0_base, D_base, r0_base, theta0_base, n)
    idx2 = np.unravel_index(np.argmax(mag2), mag2.shape)
    r_hat = r_grid[idx2[0]]
    theta_hat = theta_grid[idx2[1]]
    est2 = np.array([r_hat * np.sin(theta_hat), r_hat * np.cos(theta_hat)])
    err2 = np.sqrt(np.sum((est2 - target)**2))
    profile2 = mag2[idx2[0], :]
    peak2 = np.max(profile2)
    res2 = np.sum(profile2 > peak2/2) * (theta_grid[1] - theta_grid[0]) * r0_base
    err2_N.append(err2); res2_N.append(res2)

theta_vals = np.linspace(20, 160, 8)
err1_theta, err2_theta, res1_theta, res2_theta = [], [], [], []
for th in theta_vals:
    mag1, _, _, target = simulate_method1(f0_base, D_base, r0_base, th, N_base)
    idx1 = np.unravel_index(np.argmax(mag1), mag1.shape)
    est1 = np.array(x_grid[idx1[1]], z_grid[idx1[0]])
    err1 = np.sqrt(np.sum((est1 - target)**2))
    profile1 = mag1[idx1[0], :]
    peak1 = np.max(profile1)
    res1 = np.sum(profile1 > peak1/2) * (x_grid[1] - x_grid[0])
    err1_theta.append(err1); res1_theta.append(res1)

    mag2, theta_grid, r_grid, _ = simulate_method2(f0_base, D_base, r0_base, th, N_base)
    idx2 = np.unravel_index(np.argmax(mag2), mag2.shape)

```

```

    r_hat = r_grid[idx2[0]]
    theta_hat = theta_grid[idx2[1]]
    est2 = np.array([r_hat * np.sin(theta_hat), r_hat * np.cos(theta_hat)])
    err2 = np.sqrt(np.sum((est2 - target)**2))
    profile2 = mag2[idx2[0], :]
    peak2 = np.max(profile2)
    res2 = np.sum(profile2 > peak2/2) * (theta_grid[1] - theta_grid[0]) * r0_base
    err2_theta.append(err2); res2_theta.append(res2)

fig, axs = plt.subplots(2,2, figsize=(12,9))
fig.suptitle("Part (d) - Accuracy Analysis", fontsize=14)
axs[0,0].plot(D_lambda_vals, err1_D, 'o-b', label="Method 1 (Mono)")
axs[0,0].plot(D_lambda_vals, err2_D, 's-r', label="Method 2 (Polar)")
axs[0,0].set_xlabel("Separation D (lambda)"); axs[0,0].set_ylabel("Error (m)"); axs[0,0].legend();
axs[0,0].grid(alpha=0.3)
axs[0,1].plot(bw_frac, err1_B, 'o-b'); axs[0,1].plot(bw_frac, err2_B, 's-r')
axs[0,1].set_xlabel("Bandwidth (% f0)"); axs[0,1].grid(alpha=0.3)
axs[1,0].plot(N_vals, err1_N, 'o-b'); axs[1,0].plot(N_vals, err2_N, 's-r')
axs[1,0].set_xlabel("Steps N"); axs[1,0].grid(alpha=0.3)
axs[1,1].plot(theta_vals, err1_theta, 'o-b'); axs[1,1].plot(theta_vals, err2_theta, 's-r')
axs[1,1].set_xlabel("Bearing (deg)"); axs[1,1].grid(alpha=0.3)
plt.tight_layout()
plt.savefig(out_dir/"Part_d_Accuracy_Analysis.png", dpi=300); plt.close()

fig, axs = plt.subplots(2,2, figsize=(12,9))
fig.suptitle("Part (d) - Resolution Analysis", fontsize=14)
axs[0,0].plot(D_lambda_vals, res1_D, 'o-b', label="Method 1 (Mono)")
axs[0,0].plot(D_lambda_vals, res2_D, 's-r', label="Method 2 (Polar)")
axs[0,0].set_xlabel("Separation D (lambda)"); axs[0,0].set_ylabel("Resolution Cell Size (m)");
axs[0,0].legend(); axs[0,0].grid(alpha=0.3)
axs[0,1].plot(bw_frac, res1_B, 'o-b'); axs[0,1].plot(bw_frac, res2_B, 's-r')
axs[0,1].set_xlabel("Bandwidth (% f0)"); axs[0,1].grid(alpha=0.3)
axs[1,0].plot(N_vals, res1_N, 'o-b'); axs[1,0].plot(N_vals, res2_N, 's-r')
axs[1,0].set_xlabel("Steps N"); axs[1,0].grid(alpha=0.3)
axs[1,1].plot(theta_vals, res1_theta, 'o-b'); axs[1,1].plot(theta_vals, res2_theta, 's-r')
axs[1,1].set_xlabel("Bearing (deg)"); axs[1,1].grid(alpha=0.3)
plt.tight_layout()
plt.savefig(out_dir/"Part_d_Resolution_Analysis.png", dpi=300); plt.close()

```

2) Plot.py

```

import numpy as np
import matplotlib.pyplot as plt

files = [
    ("sweep_D_method1.txt", "D (units of λ0)", "Error vs D Method1"),

```

```

("sweep_B_method1.txt", "B=f0 (GHz)", "Error vs Bandwidth Method1"),
("sweep_r0_method1.txt", "r0 (m)", "Error vs r0 Method1"),
("sweep_theta_method1.txt", "theta0 (deg)", "Error vs theta0 Method1"),
("sweep_N_method1.txt", "N", "Error vs N Method1"),
("sweep_D_method2.txt", "D (units of  $\lambda_0$ )", "Error vs D Method2"),
("sweep_B_method2.txt", "B=f0 (GHz)", "Error vs Bandwidth Method2"),
("sweep_r0_method2.txt", "r0 (m)", "Error vs r0 Method2"),
("sweep_theta_method2.txt", "theta0 (deg)", "Error vs theta0 Method2"),
("sweep_N_method2.txt", "N", "Error vs N Method2"),
]
for fname, xlabel, title in files:
    try:
        data = np.genfromtxt(f"assignment8_results/{fname}", comments="#", dtype=float,
skip_header=2)
        x = data[:,0]
        err = data[:,1]
        plt.figure(figsize=(6,4))
        plt.plot(x, err, marker='o', linewidth=2)
        plt.grid(True)
        plt.xlabel(xlabel)
        plt.ylabel("Localization Error (m)")
        plt.title(title)
        plt.tight_layout()
        plt.savefig(f"assignment8_results/{title.replace(' ', '_')}.png", dpi=200)
        plt.close()
        print(f"Generated: {title.replace(' ', '_')}.png")
    except Exception as e:
        print(f"Error processing {fname}: {e}")
print("All plots generated!")

```